

Relatório Estágio Curricular



Curso Técnico Superior Profissional - Robótica e Inteligência Artificial

Infraestruturas de Portugal - Campus do Pragal

Eliezer Carvalho 2024392

Professor Acompanhante: Francisco Gonçalo

Orientador: António Simões

Maio 2026 - Outubro 2026

Agradecimentos

Antes de tudo, quero agradecer à Infraestruturas de Portugal e ao meu mentor António Simões por me terem dado uma oportunidade.

Resumo

Este relatório descreve o trabalho realizado durante o estágio curricular de quatro meses na empresa Infraestruturas de Portugal, na Direção de Acessibilidade, Telemática e ITS (DAT). O principal objetivo do estágio foi a exploração e o desenvolvimento de soluções baseadas em modelos de Inteligência Artificial (IA), com especial foco em modelos de linguagem e na sua aplicação em diferentes contextos empresariais.

Num primeiro momento, começou-se por explorar e experimentar as famílias dos modelos de linguagem tais como os Large Language Models (LLM's) e os Small Language Models (SLM's), dedicando-se especial atenção às técnicas e ferramentas associadas à sua execução e utilização. Foram estudados aspetos relacionados com a inferência de modelos, nomeadamente as fases de Prefill e Decode, a utilização da memória ao longo do tempo, quantização dos modelos e diferentes motores de inferência.

Num segundo momento, foram exploradas práticas de Context Engineering e Generation Synthetic Data, consideradas fulcrais para o desenvolvimento e a avaliação de sistemas baseados em modelos de linguagem. Foram apresentados materiais de apoio destinados a facilitar o entendimento de ambas as práticas.

O trabalho desenvolvido resultou, posteriormente, na construção de dois sistemas principais:

O primeiro sistema, baseado em Retrieval Augmented Generation (RAG), teve como objetivo fornecer informação contextual exterior aos modelos de linguagem. Foi construída uma versão única deste sistema e foi apresentada uma avaliação com base em diferentes métricas de recuperação e geração com o objetivo de medir a performance do sistema construído.

O segundo sistema visou a auditoria de transcrições de áudios de chamadas telefónicas. Este sistema integrou modelos de Automatic Speech Recognition (ASR) para a transcrição dos ficheiros de áudio e modelos de linguagem para a posterior auditoria das transcrições, de acordo com o contexto definido pelo utilizador. Ao longo do estágio, o projeto evoluiu, tendo sido desenvolvidas duas versões finais. Apresentámos benchmarks com diferentes configurações de transcrição e com técnicas de pré-processamento de áudio. Foi adicionada uma interface conversacional que permite fornecer informações armazenadas na base de dados como contexto a um modelo de linguagem através de uma Semantic Layer. Esta Semantic Layer foi também testada e avaliada usando benchmarks.

Durante o estágio, foi dada especial importância à documentação, investigação e avaliação das soluções desenvolvidas, o que permitiu consolidar os conhecimentos técnicos adquiridos durante a formação académica e desenvolver novas competências relacionadas com o desenvolvimento de sistemas de inteligência artificial num contexto profissional.

Índice Geral

Agradecimentos	1
Resumo	2
Índice Geral	4
Índice de Tabelas	6
Índice de Figuras	7
Siglas, Abreviaturas e Significados	8
Parte I - Introdução	9
1.1 Infraestruturas de Portugal	9
1.2 História	10
Estradas de Portugal	10
Rede Ferroviária Nacional	10
1.3 Organograma	11
1.4 Quem Somos - Infraestruturas de Portugal	12
Missão	12
Visão	12
Valores	12
Parte II - Prólogo Estágio Curricular	13
2.1 Inteligência Artificial: ainda um admirável mundo novo ?	13
2.2 Enquadramento do Estágio	15
Parte III - Fundamento Estágio Curricular	16
3.1 Uma Análise Tridimensional para a Hospedagem de LLM's	16
3.1.1 Fase de Investigação	16
3.1.2 Fase de Aplicação	19
• Performance	19
• Tempo	20
• Memória	21
3.1.3 Conclusão e Futuro	22
3.2 Context Engineering	23
3.2.1 Fase de Investigação	23
3.2.2 Fase de Aplicação	24
3.2.3 Conclusão e Futuro	25
3.3 Synthetic Data	26
3.3.1 Fase de Investigação	26
3.3.2 Fase de Aplicação	28
3.3.3 Conclusão e Futuro	30
Parte IV - Sistemas Estágio Curricular	31
4.1 How To RAG (Retrieval Augmented Generation)	31
4.1.1 Fase de Investigação	31
4.1.2 Fase de Aplicação	34
Arquitetura do Sistema	34
Avaliação do Sistema	35
Dados e integração com as fases anteriores	36
Demonstração e Documentação	37
4.1.3 Conclusão e Futuro	38
4.2 Sistema Assobio	39
4.2.1 Sistema Assobio - Versão 1	39
4.2.2 Sistema Assobio - Versão 1 Fase de Investigação	41
4.2.3 Sistema Assobio - Versão 1 Fase de Aplicação	42

4.2.4 Sistema Assobio - Versão 2	44
4.2.5 Sistema Assobio - Versão 2 Fase de Investigação	45
4.2.6 Sistema Assobio - Versão 2 Fase de Aplicação	46
Parte V - Conclusão	47
Parte VI - Bibliografia	48

Índice de Tabelas

[Tabela 1 - Siglas, Abreviaturas e Significados](#)

[Tabela 2 - Tamanho de um modelo de Linguagem de 7B de parâmetros em diferentes formatos de Quantização](#)

[Tabela 3 - Prefill vs Decode](#)

[Tabela 4 - Time to First Token vs Latência](#)

Índice de Figuras

[Figura 1 - Infraestruturas de Portugal](#)

[Figura 2 - Organograma Infraestruturas de Portugal](#)

Siglas, Abreviaturas e Significados

DAT	Direção de Acessibilidade e Telemática	Direção onde o estagiário esteve inserido dentro da Infraestruturas de Portugal
IA	Inteligência Artificial	Área base do estágio

LLM's	Large Language Models	Modelos de Linguagem de grande escala
SLM's	Small Language Models	Modelos de Linguagem de pequena escala
RAG	Retrieval Augmented Generation	Técnica para recuperação de contexto
ASR	Automatic Speech Recognition	Modelos de Inteligência Artificial orientados para a área de áudio
CPU	Unidade Central de Processamento	Hardware pai, mãe, filho, neto dos computadores modernos
GPU	Unidades de Processamento Gráfico	Hardware importante na vertente gráfica dos computadores modernos
TTFT	Time To First Token	Tempo até a primeira iteração da fase de Decode estar concluída

Tabela 1 - Siglas, Abreviaturas e Significados

Parte I - Introdução

1.1 Infraestruturas de Portugal

Nos últimos anos, a IP evoluiu com um foco mais alargado na gestão da mobilidade multimodal e na vasta rede de infraestruturas sob a sua responsabilidade, sob o

lema "Ligamos Destinos", com um forte impacto no desenvolvimento económico e social do país.

Nos termos do Contrato de Concessão Geral da rede rodoviária nacional, celebrado com o Estado, a IP é responsável pela gestão de infraestruturas rodoviárias e pela prestação do serviço público de gestão da infraestrutura integrante da Rede Ferroviária Nacional (RFN), em regime de delegação de competências do Estado Português, através da execução de um Contrato Programa para o setor ferroviário.

Neste contexto, a IP destaca-se como uma empresa única a nível nacional e internacional, combinando um know-how singular, graças à experiência e competência dos seus colaboradores, com uma grande abertura e apetência para a inovação, fator decisivo face à atual evolução permanente dos serviços de mobilidade.

A IP, que tem o Estado Português como único acionista, está sujeita à tutela do Ministério das Infraestruturas e Habitação e do Ministério das Finanças. [\[1\]](#)



Figura 1 - Infraestruturas de Portugal

1.2 História

Estradas de Portugal

A Estradas de Portugal, S.A., foi uma sociedade anónima de capitais exclusivamente públicos, criada no dia 21 de Dezembro de 2004 sob a

denominação "EP - Estradas de Portugal, EPE", com o objetivo de administrar as estradas nacionais em Portugal.

A empresa sucedeu à JAE - Junta Autónoma das Estradas, fundada em 20 de Julho de 1927 - e ao IEP - Instituto das Estradas de Portugal, criado em 25 de Junho de 1999. A EP atuava nas áreas do financiamento, da conservação, da exploração, da requalificação e do alargamento da rede rodoviária nacional. [2]

Rede Ferroviária Nacional

A REFER (Rede Ferroviária Nacional, E.P.) foi criada em 1997 pelo Decreto-Lei n.º 104/97, de 29 de abril, como empresa pública responsável pela gestão, renovação, construção e conservação da infraestrutura ferroviária nacional.

No entanto, a REFER não podia explorar quaisquer serviços ferroviários, a fim de manter uma total isenção em relação aos diversos operadores ferroviários e assegurar uma melhor harmonização com as iniciativas privadas neste setor. A REFER estava sujeita à tutela dos ministros responsáveis pelas áreas das finanças e dos transportes.

Por meio do Decreto-Lei n.º 91/2015, de 29 de maio, com efeitos a partir de 1 de junho de 2015, a REFER incorporou a Estradas de Portugal (EP) por fusão, tendo-se convertido em sociedade anónima (S.A.) e passado a denominar-se **Infraestruturas de Portugal, S.A.** [3]

1.3 Organograma

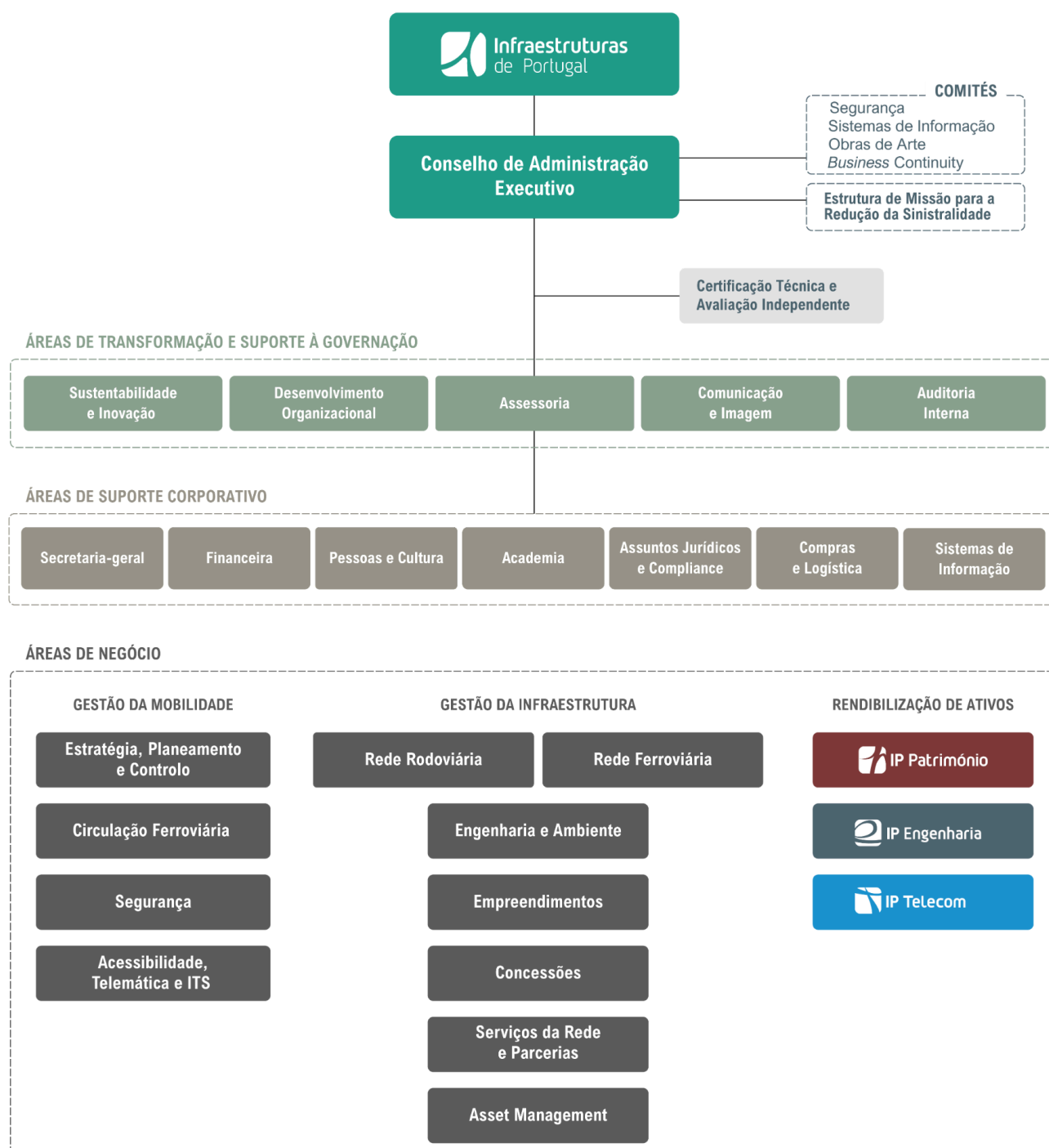


Figura 2 - Organograma Infraestruturas de Portugal

1.4 Quem Somos - Infraestruturas de Portugal

Missão

A IP tem como missão conceber, projetar, construir, financiar, conservar, explorar, reabilitar, ampliar e modernizar as redes rodoviárias e ferroviárias nacionais, incluindo o comando e o controlo da circulação ferroviária. A IP é ainda responsável pela conceção, projeto, construção, exploração, operação e manutenção, em regime de concessão com possibilidade de subconcessão, do sistema de cabos submarinos de comunicações eletrónicas entre o território continental de Portugal e os arquipélagos dos Açores e da Madeira. [4]

Visão

Posicionar a Infraestruturas de Portugal como gestora de mobilidade multimodal, potenciando o asset management e garantindo a prestação de um serviço seguro, eficiente e sustentável, valorizado pela rentabilização de ativos complementares. [5]

Valores

Ética, a organização orienta a sua ação de acordo com os princípios da transparência, da boa-fé e da honestidade.

Segurança é igualmente um princípio fundamental que orienta a atividade para a proteção da vida e da integridade física das pessoas, sendo um dos atributos mais relevantes do serviço prestado.

Sustentabilidade está presente na orientação da organização para um desenvolvimento equilibrado das suas dimensões económica, social e ambiental. [6]

Parte II - Prólogo Estágio Curricular

2.1 Inteligência Artificial: ainda um admirável mundo novo ?

Início este relatório de estágio curricular abordando o que considero ser uma das questões mais importantes para compreender o trabalho desenvolvido ao longo destes quatro meses: afinal, qual é o verdadeiro valor da IA e que direção deverá tomar no contexto empresarial ?

Atualmente, todos ouvimos falar de IA. Referimo-nos a modelos de linguagem, geração de texto e imagens, assistentes virtuais, automação, agentes etc... No entanto, apesar da crescente presença desta tecnologia no nosso quotidiano, penso que há uma diferença significativa entre usar IA e compreender verdadeiramente como estes sistemas funcionam, quais as suas limitações e onde podem efetivamente acrescentar valor. A IA é claramente uma doença que faz muita gente perder o discernimento.

Na minha opinião, a atual evolução da Inteligência Artificial pode ser comparada, em termos de dimensão e potencial transformador, a outras grandes mudanças tecnológicas que marcaram a história da humanidade. A Revolução Industrial alterou profundamente a forma como produzimos bens e organizamos o trabalho, ao passo que a eletrificação do mundo transformou praticamente todos os setores da sociedade. A Inteligência Artificial representa uma transformação de dimensão semelhante. De longe falo de substituir todas as atividades humanas, mas sublinho a sua capacidade de alterar a forma como certas tarefas são executadas, a forma como a informação é processada e a forma como as organizações interagem com os seus dados.

No entanto, com um grande poder também surge uma grande responsabilidade. Essa responsabilidade é individual, mas adquire uma dimensão ainda maior no caso das organizações. Uma empresa que decide introduzir Inteligência Artificial nos seus processos não está apenas a adotar uma nova ferramenta. Pode estar a introduzir sistemas capazes de processar informação sensível, interagir com dados internos, apoiar a tomada de decisões e alterar processos anteriormente dependentes de intervenção humana.

Considero que é aqui que surge uma espécie de relação de "amor e ódio" entre as organizações e a Inteligência Artificial. Por um lado, há uma grande expectativa em relação ao potencial desta tecnologia para aumentar a produtividade. Na outra face da moeda, não se podem ignorar os desafios associados: a proteção e a qualidade dos dados, a segurança das infraestruturas, os custos computacionais, a privacidade, a fiabilidade das respostas, a integração com os sistemas existentes, a

dependência de fornecedores externos e, sobretudo, a necessidade de compreender o que está efetivamente a ser colocado em produção.

Esta questão é particularmente relevante no contexto de organizações de grande dimensão e importância estratégica. No âmbito do meu estágio, deparei-me com uma realidade que considero ilustrativa deste desafio: mesmo numa das maiores empresas públicas do Estado Português, **a adoção de processos baseados em IA executados diretamente sobre infraestruturas próprias ainda se encontrava numa fase crisálida**. Isso não significa que não houvesse interesse ou investigação em torno da tecnologia. Significa antes que transformar experiências com modelos de IA em processos internos controlados e preparados para utilização empresarial envolve um conjunto de desafios que vão muito além da simples adoção de um modelo open-source.

Este exemplo ajuda a introduzir uma das questões que estive na origem do meu estágio. Se uma organização possui informação interna que não pode ou não deve ser enviada para serviços externos, como pode tirar partido das capacidades atuais dos modelos de Inteligência Artificial ? Se determinados processos são repetitivos, demorados e dispendiosos em termos de recursos humanos, será possível utilizar modelos de linguagem ou outras técnicas de IA e Machine Learning para os apoiar ou automatizar, mantendo simultaneamente o controlo sobre os dados e a infraestrutura ?

É aqui que eu entro.

2.2 Enquadramento do Estágio

Nas primeiras semanas de estágio, foram realizadas algumas reuniões com o orientador para definir a direção e os objetivos a alcançar ao longo dos quatro meses. Desde cedo ficou estabelecido um princípio que esteve presente em praticamente todo o trabalho desenvolvido: "A tecnologia tem de estar do nosso lado, isolada da internet, e temos de ter o controlo total."

Esta frase, que surgiu de forma natural durante as primeiras reuniões, acabou por representar a base sobre a qual assenta grande parte do meu trabalho. Mais do que uma preferência tecnológica, este princípio refletia a necessidade de compreender e controlar todo o ciclo de utilização da Inteligência Artificial numa organização com infraestruturas críticas. *Neste relatório não me vou cansar de referir a dimensão da Infraestruturas de Portugal, que acho que só por dentro é que se tem a bigger picture.* Desde os dados utilizados até aos resultados produzidos, passando pelos modelos e pela infraestrutura necessária para os executar, procurou-se privilegiar uma abordagem em que a organização mantivesse o maior controlo possível sobre a tecnologia.

Este princípio teve também uma influência significativa na forma como encarei o estágio. O objetivo não era apenas encontrar um modelo capaz de resolver um problema específico e aplicá-lo imediatamente. Era necessário compreender o que estava por trás dessa solução, quais os seus requisitos e limitações, e como poderia ser integrada num ambiente empresarial. Aqui cheguei à conclusão que este estágio encaixava que nem uma luva no meu perfil de pessoa técnica, que gosta de aprofundar e de perguntar “porquês” só porque sim.

Desde o início, ficou igualmente definido que, apesar do estágio estar fortemente orientado para a área da IA, não deveria haver uma visão limitada da tecnologia. A Inteligência Artificial não passou a ser a única ferramenta disponível para resolver problemas tecnológicos. Sempre que uma abordagem baseada em Machine Learning, em métodos tradicionais de processamento de dados ou em princípios de engenharia de software fosse mais adequada, essas alternativas também deveriam ser consideradas.

Esta ideia levantou-se várias vezes ao longo do estágio. Enquanto alguns problemas beneficiaram diretamente da utilização de LLM's, outros exigiram componentes mais tradicionais. Foi importante não enfiarmos a cabeça na areia.

A IA sem um sistema ao seu redor, torna-se uma tecnologia banal.

Parte III - Fundamento Estágio Curricular

3.1 Uma Análise Tridimensional para a Hospedagem de LLM's

3.1.1 Fase de Investigação

No início do estágio, uma das primeiras dúvidas que me surgiu foi compreender como poderia utilizar efetivamente os modelos de linguagem. Atualmente, os frontier Language Models estão disponíveis a um clique de distância e escondem grande parte da complexidade associada à sua execução. Para o utilizador final, esta abstração é extremamente conveniente. No contexto deste estágio, a abstração tinha que ser metida de lado, para alcançar o “sonho da soberania sobre os modelos”.

Comecei a pesquisar as diferentes opções existentes para obter e executar modelos de Inteligência Artificial. Uma das plataformas que se destacou rapidamente foi a **Hugging Face** [7], que se tornou um dos principais ecossistemas para o desenvolvimento e partilha de modelos de IA. Além de disponibilizar um vasto conjunto de modelos, a plataforma oferece também datasets, bibliotecas e ferramentas que facilitam o desenvolvimento e a investigação com diferentes tipos de modelos [8]. A utilização deste ecossistema permitiu-nos superar uma primeira barreira. Imediatamente surgiu outra limitação.

As primeiras experiências eram realizadas com recurso à CPU (Unidade Central de Processamento), o que suscitou uma questão importante em relação ao desempenho da inferência.

Os modelos de linguagem modernos baseiam grande parte do seu funcionamento em matrix multiplications de grande dimensão, que são particularmente relevantes no mecanismo de *Attention* utilizado pelas arquiteturas Transformer [9]. Estas operações apresentam um elevado grau de paralelismo. As CPU são processadores de propósito geral, concebidos para executar uma grande variedade de tarefas, ao passo que as GPU (Unidades de Processamento Gráfico) foram desenvolvidas com uma arquitetura especialmente adequada à execução de grandes quantidades de operações em paralelo.

A importância das GPU para o desenvolvimento moderno de modelos de Inteligência Artificial não é recente. Um dos momentos mais marcantes ocorreu em 2012 com o trabalho de Alex Krizhevsky, Ilya Sutskever e Geoffrey Hinton que, através da Rede Neural AlexNet, demonstraram de forma muito expressiva as vantagens da utilização de GPUs no treino de Deep Neural Networks [10] [11].

No âmbito deste estágio, esta questão deixou de ser apenas teórica. **Em colaboração com o orientador e a Infraestruturas de Portugal, foi possível disponibilizar um servidor equipado com uma placa gráfica NVIDIA RTX A4000, com 16 GB de memória de vídeo (VRAM).** O acesso a este equipamento alterou significativamente o cenário inicial, permitindo transitar da execução exclusivamente em CPU para a exploração da inferência acelerada por GPU.

A partir deste momento, tornou-se necessário compreender melhor o software responsável por estabelecer a ligação entre os modelos e o hardware disponível. Uma das ferramentas que já conhecia era o **PyTorch** [12], uma estrutura de aprendizagem automática que disponibiliza abstrações em Python para a construção e execução de modelos, incluindo suporte para aceleração por meio de GPUs da NVIDIA. Por trás desta aceleração está o ecossistema CUDA [13], desenvolvido pela NVIDIA. CUDA permite a execução de código e operações computacionais diretamente na GPU, por meio de núcleos especializados. Neste contexto, PyTorch funciona como uma camada de abstração que permite utilizar estas capacidades através de uma interface high level, evitando que o programador tenha de implementar diretamente a maior parte das operações de nível inferior.

Foi nesta fase que surgiu uma questão que se revelou muito importante para o resto da minha investigação: *dado que as GPU apresentam vantagens tão significativas na execução de modelos de linguagem, como podem estes modelos ser utilizados por organizações ou indivíduos sem acesso a hardware especializado ?* Embora esta questão não representasse uma limitação direta durante o estágio, uma vez que foi possível recorrer à infraestrutura disponibilizada pela organização, considerei importante compreender quais as alternativas existentes. A minha realidade não era a de todos os indivíduos e nem de todas as empresas.

Esta investigação levou-me ao conceito de motores de inferência, ou Inference Engines. Um motor de inferência é responsável por executar um modelo de forma otimizada, aproveitando ao máximo o hardware disponível. A inferência de um modelo de linguagem depende não só do próprio modelo, mas também da forma como as operações são organizadas, da gestão da memória, da utilização do hardware e de várias otimizações específicas da fase de inferência, pelo que estes sistemas assumem particular importância.

Entre as soluções exploradas, destacou-se o **llama.cpp** [14], um projeto amplamente utilizado no ecossistema open source para executar modelos de linguagem localmente, com especial foco na eficiência e na utilização da CPU, embora também ofereça suporte para diferentes formas de aceleração por GPU. Uma das características que tornou o llama.cpp particularmente relevante para esta investigação foi a utilização do formato GGUF associado à execução de modelos

quantizados. Foi aqui que a quantização tornou-se outro dos conceitos importantes explorados durante esta fase do estágio.

A Quantização consiste, de forma simplificada, em representar os pesos de um modelo com uma quantidade reduzida de bits. Os modelos cujos pesos são originalmente representados com maior precisão podem ser convertidos, através de técnicas de quantização, para representações com menor precisão, o que reduz significativamente a memória necessária para os armazenar e executar. Esta possibilidade é particularmente relevante quando se pretende executar modelos localmente. A capacidade de reduzir a memória necessária para carregar um modelo faz toda a diferença na execução de um modelo específico num determinado equipamento. Durante o estágio, esta questão tornou-se especialmente pertinente, dado o limite concreto de 16 GB de VRAM na placa gráfica NVIDIA RTX A4000.

Formato da Quantização	Tamanho
16 Bits	14 GB
8 Bits	7 GB
4 Bits	3.5 GB

Tabela 2 - Tamanho de um modelo de Linguagem de 7B de parâmetros em diferentes formatos de Quantização

A investigação realizada nesta primeira fase permitiu-me perceber que a escolha do modelo não pode ser dissociada da infraestrutura em que será executado. O número de parâmetros, a precisão dos pesos, a quantidade de memória disponível, o tipo de processador, a capacidade da GPU e o motor de inferência utilizado em si influenciam diretamente a possibilidade e o desempenho da execução.

A partir daqui, passámos de uma fase de investigação a uma questão mais concreta: *como podemos avaliar o desempenho de diferentes modelos quando executados em hardware específico e, simultaneamente, compreender os compromissos existentes entre capacidade computacional, tempo de execução e memória ?*

3.1.2 Fase de Aplicação

Propus uma Análise Tridimensional para a Hospedagem de LLM's baseada nas seguintes dimensões:

- Performance

Esta dimensão teve como objetivo compreender se os recursos computacionais disponíveis eram suficientes para executar um modelo de linguagem específico e, simultaneamente, perceber como diferentes configurações de hardware poderiam influenciar o desempenho na fase de inferência.

Para este efeito, foram consideradas as duas principais fases da geração de uma resposta por parte de um LLM:

Prefill	Decode
Corresponde ao momento em que o modelo processa e interpreta o prompt fornecido pelo utilizador. Nesta fase, todos os tokens de entrada são submetidos a uma análise, de modo a construir o contexto necessário à geração da resposta.	O modelo gera output tokens sequencialmente, com base em todos os tokens anteriores (incluindo o prompt e quaisquer output tokens previamente gerados). Para cada novo token, o modelo executa uma forward pass e utiliza mecanismos de <i>Attention</i> para considerar todo o contexto acumulado da conversa.

Tabela 3 - Prefill vs Decode

Para ambas as fases, calculou-se o throughput, utilizando como unidade tokens/s (tokens por segundo).

Estas métricas permitem quantificar a quantidade de tokens processados pelo modelo por unidade de tempo e, conseqüentemente, comparar o comportamento das diferentes configurações utilizadas. O modelo de referência selecionado foi o **Qwen3 4B Base** [15] e foram realizados testes com diferentes tamanhos de contexto: 2000 e 4000 tokens. É claro que tamanhos de contexto diferentes correspondem a performances diferentes.

Observou-se que, quanto maior for o contexto fornecido a um modelo, maior será a quantidade de informação que este terá de processar durante a fase de Prefill. Isso resulta num aumento do tempo necessário para processar o input.

Durante a fase de Decode, o comportamento é diferente. Nesta fase, o modelo gera os output tokens de forma autoregressiva, recorrendo à informação previamente processada e armazenada no **KV Cache** (Key-Value Cache). Um contexto inicial maior implica também um KV Cache de maior dimensão.

Concluiu-se que, enquanto a fase de Prefill é predominantemente *Compute-Bound*, ou seja, o seu desempenho está fortemente condicionado pela capacidade de processamento do hardware, a fase de Decode apresenta um comportamento predominantemente *Memory-Bound*. Isto porque durante a fase de Decode, o modelo realiza uma nova forward pass a cada token gerado, sendo necessário aceder repetidamente aos pesos do modelo e às informações armazenadas no KV Cache. A velocidade com que estes dados podem ser transferidos entre a memória e as unidades de processamento é um fator empírico.

- Tempo

A segunda dimensão desta análise é o Tempo. Esta dimensão tem dois objetivos distintos, mas complementares: compreender o impacto do tempo de execução na experiência do utilizador e medir o desempenho na vertente Tempo de um modelo quando executado num dispositivo de hardware específico.

Com estes objetivos em mente foram calculadas duas métricas:

Time to First Token	Latência
Corresponde ao tempo decorrido entre a receção da prompt e a geração do primeiro token de saída. Esta métrica abrange o processamento inicial necessário para preparar a resposta, nomeadamente a tokenização, a fase de Prefill e a primeira iteração do Decode, que é responsável pela geração do primeiro token. O TTFT é particularmente relevante para avaliar a rapidez com que o utilizador começa a receber uma resposta.	Latência é a métrica que avalia a experiência do utilizador. Mede o tempo decorrido entre o envio do prompt e a geração de uma resposta completa. Esta métrica abarca todas as fases do pipeline de inferência, incluindo a Tokenização, o Prefill, o Decode e a geração total do output, refletindo, deste modo, o comportamento end-to-end do sistema.

Tabela 4 - Time to First Token vs Latência

Para ambas as fases, a unidade é segundos. Menor é melhor.

Tal como na dimensão anterior, foram realizados testes com o mesmo modelo de referência (Qwen3 4B Base) e diferentes tamanhos de contexto.

Observou-se que à medida que o número de tokens de contexto aumenta, verifica-se um aumento do tempo necessário para processar a instrução e gerar a resposta.

Este comportamento é particularmente evidente no TTFT, dado que o aumento do contexto implica uma quantidade maior de informação para processar durante a fase de Prefill, antes de poder ser produzido o primeiro token.

A Latência também tende a aumentar com o tamanho do contexto, visto o sistema partir de um estado no qual é necessário processar uma quantidade de informação superior antes de iniciar a geração.

Concluiu-se que o tempo necessário para surgir o primeiro token e o tempo total para concluir a resposta são dimensões complementares que ajudam a compreender o comportamento de um sistema de inferência.

- **Memória**

A terceira e última dimensão da análise corresponde à Memória. O objetivo desta dimensão é avaliar a quantidade de memória necessária para executar um modelo de linguagem, bem como compreender como essa necessidade varia à medida que o contexto aumenta.

Ao analisar a memória necessária para executar um LLM, não basta considerar apenas o tamanho dos respectivos parâmetros. Durante a inferência, existem outros componentes que ocupam memória e variam dinamicamente de acordo com a utilização do modelo. Um dos mais relevantes é o já mencionado, KV Cache.

A investigação realizada permitiu observar que, à medida que o contexto fornecido ao modelo aumenta, também aumenta a memória necessária para a execução da inferência. Mantendo a arquitetura e os restantes parâmetros constantes, a quantidade de memória ocupada pelo KV Cache tem uma relação aproximadamente linear com o número de tokens do contexto.

Esta relação tornou-se particularmente relevante quando se analisou o impacto da quantização. Ao representar os pesos do modelo com uma quantidade reduzida de bits, a quantização permite diminuir significativamente a memória necessária para os armazenar.

Concluiu-se que a quantização é uma técnica que pode e deve ser executada aos modelos de linguagem, independentemente do hardware disponível, pois permite reduzir significativamente os requisitos de memória e melhorar a eficiência da inferência com um custo de qualidade mínimo.

3.1.3 Conclusão e Futuro

A primeira fase do estágio foi muito importante, pois procurou introduzir um procedimento que permitisse analisar de forma mais estruturada os recursos necessários para executar modelos de linguagem. Com base nas três dimensões definidas - *Performance*, *Tempo* e *Memória* -, tentei compreender não só se um modelo específico poderia ser executado num determinado dispositivo, como também em que condições essa execução poderia ser considerada eficiente e viável.

A nível pessoal, esta fase também foi importante, pois alterou a minha perceção inicial sobre o hardware necessário para hospedar modelos de linguagem. Inicialmente, a capacidade de memória da GPU parecia ser o fator mais importante para determinar a possibilidade de execução de um modelo. No entanto, à medida que a análise progredia, percebi que esta é apenas uma das dimensões a considerar. A capacidade de processamento do dispositivo e a velocidade de comunicação entre a memória e as unidades de processamento também têm um impacto significativo no desempenho da inferência. Isto serve tanto para empresas que já tenham hardware disponível em casa e queiram analisar a performance de um modelo, como para empresas que estejam à procura de hardware para começar a hospedar os seus próprios modelos.

Outro aspeto relevante foi compreender que a evolução dos motores de inferência tornou a execução local de modelos de linguagem muito mais acessível. Ferramentas como o llama.cpp permitem executar modelos em diferentes tipos de hardware, incluindo CPU e GPU, reduzindo a dependência de infraestruturas dispendiosas. Naturalmente, o desempenho obtido continua a depender das características do hardware, do modelo utilizado, da quantização e da configuração da inferência. Ainda assim, esta experiência permitiu-me compreender que a execução local de LLM não está necessariamente limitada a infraestruturas de grande dimensão.

No que se refere a perspetivas futuras, esta análise poderia ser aprofundada. Uma das ideias mais cobiçadas seria estudar a aplicação e otimização da inferência em linguagens low level, possibilitando um controlo mais rigoroso das operações específicas e da utilização dos recursos computacionais. Esta abordagem também possibilitaria a recolha de métricas mais detalhadas, permitindo compreender com maior precisão onde se encontram os principais custos da inferência. Considero esta uma limitação técnica da minha experiência atual, mas também uma das áreas que pretendo aprofundar no futuro.

3.2 Context Engineering

3.2.1 Fase de Investigação

Ao longo do estágio, uma das ideias mais presentes foi a necessidade de garantir que os modelos de linguagem tinham capacidade para executar o que lhes era pedido, respeitando simultaneamente regras e restrições bem definidas. Esta questão é particularmente relevante para uma organização, visto a soberania sobre um sistema de inteligência artificial não se *limitar ao controlo da sua execução, mas estender-se também ao controlo do seu comportamento*.

Como os modelos de linguagem são sistemas que interagem principalmente através da linguagem, a melhor maneira de forçar um modelo a ter o comportamento que desejamos é com... linguagem.

Surgiu o estudo da Engenharia de Contexto (Context Engineering), que procura compreender como o contexto fornecido a um modelo pode ser estruturado para o orientar de maneira mais consistente na realização de uma tarefa específica.

O contexto de um modelo não é apenas texto adicional fornecido juntamente com a instrução, visa mais um conjunto de instruções, regras, informação e conhecimento que definem o que o modelo deve considerar durante a execução de uma tarefa. O **Prompt** representa o pedido ou o objetivo concreto apresentado ao modelo, ao passo que o **Contexto** pode incluir as regras a serem seguidas, a informação relevante para a tarefa e as limitações impostas. Uma forma simples de compreender esta diferença é imaginar que se está a explicar a uma criança não só o que deve fazer numa determinada situação, mas também quais são as regras que deve seguir, qual é o seu papel e quais são os limites do seu comportamento.

É importante os modelos saberem o que podem fazer mas também o que NÃO podem fazer.

Inicialmente, esta área pareceu-me algo abstrata, sobretudo porque não existe uma forma única de construir o contexto ideal para todos os modelos e tarefas. No entanto, ao longo do estágio, ficou evidente que a forma como a informação é apresentada ao modelo pode afetar significativamente o seu comportamento. Um modelo pode ter capacidade para realizar uma tarefa específica e, mesmo assim, apresentar resultados inconsistentes se as instruções, as restrições ou a informação fornecida não estiverem bem estruturadas. A Engenharia de Contexto acabou por assumir um papel transversal durante o estágio.

3.2.2 Fase de Aplicação

Esta fase de aplicação não decorreu de forma tão linear como as restantes, visto que o objetivo não era construir um sistema, mas sim transformar o conhecimento adquirido numa ferramenta que pudesse ser utilizada por diferentes pessoas na organização.

Como resultado desta fase, desenvolvi um guia interno a que toda a empresa tivesse acesso e que visasse explicar, de forma simples e prática, como construir contextos mais eficazes e, consequentemente, obter respostas mais adequadas por parte dos modelos de linguagem. Procurei que o guia fosse, simultaneamente, simples e sucinto. O público-alvo principal são pessoas sem conhecimentos aprofundados na área da Inteligência Artificial. No entanto, considerei igualmente importante incluir alguns conteúdos de natureza mais técnica, de modo a tornar o documento igualmente útil para profissionais com maior experiência na área.

O guia começa por abordar o conceito de token, explicando o que representa e a sua importância para o funcionamento dos modelos de linguagem. Um dos aspetos que procurei transmitir é o de que fornecer mais informação a um modelo não significa, necessariamente, obter uma resposta melhor. O excesso de informação pode introduzir ruído.

A partir desta introdução, são apresentadas várias recomendações práticas para a construção de contextos. Estas recomendações visam auxiliar o utilizador na definição clara das responsabilidades do modelo.

3.2.3 Conclusão e Futuro

Em conclusão, considero importante salientar a relevância que a Engenharia de Contexto acabou por assumir ao longo do estágio. Embora não tenha resultado diretamente na construção de um sistema, permitiu-me introduzir conceitos que foram constantemente utilizados em diferentes tarefas. A forma como o contexto é construído tornou-se uma preocupação transversal sempre que foi necessário orientar um modelo de linguagem para uma tarefa específica.

A documentação criada teve igualmente como objetivo ir além de uma utilização específica. Os princípios apresentados podem ser aplicados a diferentes modelos de linguagem, tanto modelos de open source como frontier models, embora a sua eficácia possa variar consoante o modelo, a tarefa e o contexto de utilização.

Quanto ao futuro, há um amplo espaço para aprofundar esta área.

A Engenharia de Contexto continua a evoluir à medida que os próprios modelos de linguagem evoluem, com o desenvolvimento de novas técnicas para estruturar instruções, selecionar informação relevante e melhorar a consistência das respostas.

Uma evolução particularmente interessante seria a transição de uma abordagem exclusivamente baseada na construção manual do contexto para uma abordagem que permitisse a sua avaliação e otimização automáticas. Durante o estágio, discutiu-se a possibilidade de desenvolver um mecanismo capaz de analisar as respostas produzidas pelo modelo e de determinar se o contexto utilizado foi eficaz para a tarefa em questão. Com base nessa avaliação, o modelo poderia alterar o contexto e comparar os resultados obtidos, de modo a identificar configurações que produzissem respostas mais adequadas.

Esta abordagem poderia recorrer a técnicas de otimização automática e, consoante a formulação do problema, a métodos de Reinforcement Learning, bem como a outras estratégias de avaliação e otimização. O objetivo seria criar um ciclo contínuo entre o contexto, a resposta, a avaliação e a melhoria, de modo a permitir que a configuração do contexto evoluísse com base nos resultados obtidos.

3.3 Synthetic Data

3.3.1 Fase de Investigação

“Os dados são o novo petróleo”

Clive Humby

Machine Learning e a Inteligência Artificial assentam-se num recurso fundamental: **Dados**. Foi com esta ideia em mente que surgiu a terceira fase do estágio, dedicada ao estudo dos dados sintéticos.

Os dados sintéticos não são um conceito recente, tendo sido utilizados há vários anos em diferentes áreas de Machine Learning. De forma simplificada, os dados podem ser divididos em estruturados e não estruturados. Tradicionalmente, a utilização de dados sintéticos em Machine Learning esteve bastante associada à geração de dados estruturados, por exemplo, tabelas, registos ou exemplos destinados ao treino e avaliação de modelos.

No entanto, com a evolução das arquiteturas Transformer, em particular dos modelos de linguagem, surgiu uma nova possibilidade: a utilização destes modelos para gerar automaticamente grandes quantidades de dados estruturados e não estruturados. Foi precisamente nesta capacidade que me baseei durante esta fase do estágio.

A possibilidade de gerar dados de forma controlada pode representar uma ferramenta importante para indivíduos e organizações. Permite, por exemplo, criar exemplos para testar sistemas, aumentar conjuntos de dados, construir cenários de avaliação ou desenvolver protótipos, sem ser necessário depender exclusivamente de grandes quantidades de dados reais.

Partindo desta ideia, iniciei uma série de testes para compreender como poderia utilizar modelos de linguagem para gerar dados sintéticos. No entanto, rapidamente surgiu uma dificuldade importante. Os modelos de linguagem são sistemas estocásticos. Numa conversa normal, esta característica pode ser útil, na medida em que permite alguma diversidade nas respostas. Porém, na geração automática de dados, essa variabilidade pode tornar-se um problema.

Quando o objetivo é gerar milhares de exemplos, não é desejável que o modelo introduza constantemente informação não solicitada ou altere a estrutura esperada do resultado. A geração de um conjunto de dados com 10 000 exemplos que tivesse de ser validado e corrigido manualmente devido a informação adicional ou a formatos inválidos seria um processo pouco eficiente.

Foi neste contexto que surgiu uma das técnicas mais relevantes estudadas nesta fase: **Constraint Decoding**. Esta técnica permite restringir os resultados de um modelo de linguagem de acordo com um formato ou um conjunto de regras previamente definidos.

Para compreender o seu funcionamento, é importante recordar que, durante a geração de cada token, o modelo produz uma distribuição de probabilidades sobre os tokens que poderão ser escolhidos para continuar a sequência. O Constraint Decoding pode atuar neste processo, restringindo os tokens considerados válidos em cada fase da geração. Em termos práticos, é possível aplicar máscaras aos logits produzidos pelo modelo, de modo a impedir a seleção de tokens que não estejam em conformidade com as regras definidas. Assim, em vez de permitir que o modelo explore livremente todo o espaço de sequências possíveis, a geração fica limitada às sequências que respeitam as restrições estabelecidas. Por exemplo, se o modelo for programado para produzir um objeto JSON com campos específicos, o mecanismo de Constraint Decoding pode impedir a geração de tokens que violem a estrutura esperada. No fundo reduzimos o espaço latente do modelo de linguagem.

Esta capacidade é particularmente importante na geração de dados sintéticos, na medida em que possibilita a obtenção de resultados mais previsíveis e estruturalmente válidos, reduzindo significativamente a necessidade de processamento e limpeza manuais dos dados. Assim, o objetivo deixa de ser apenas solicitar ao modelo a produção de um formato específico, passando também a existir um mecanismo técnico que restringe o que o modelo pode efetivamente gerar.

Nada nos impede de pedir a um modelo - "Retorna o output em formato JSON" - mas isso num sistema com objetivo de gerar milhares de exemplos, não é fidedigno.

3.3.2 Fase de Aplicação

Nesta fase, foram criados dois datasets distintos para avaliar, posteriormente, um sistema RAG, o qual será abordado mais adiante neste relatório.

Foram utilizados dois modelos de linguagem de diferentes dimensões. Num primeiro momento, utilizou-se o modelo **Microsoft Phi-4** [16], com 14 mil milhões de parâmetros e quantizado a 4 bits, para gerar um dataset em formato JSON, recorrendo **apenas** às instruções definidas no contexto e sem aplicar Constraint Decoding.

Num segundo momento, utilizou-se o **Mistral 7B Instruct** [17], igualmente quantizado a 4 bits, para gerar um dataset **recorrendo** à técnica Constraint Decoding. Desta forma, foi possível comparar não só dois modelos diferentes, como também duas abordagens distintas para a geração de dados sintéticos: uma baseada exclusivamente nas instruções fornecidas ao modelo e outra que adicionava uma restrição técnica relativamente ao formato da saída. Os resultados obtidos foram dois datasets que, após avaliação humana, apresentaram qualidade suficiente para serem utilizados na fase seguinte do projeto. No entanto, foram detetadas diferenças significativas no processo de geração e validação dos mesmos.

O dataset gerado apenas com base em instruções exigiu um trabalho de limpeza e validação consideravelmente superior. Uma vez que o modelo não possuía qualquer mecanismo de bloqueio que impedisse a produção de informação fora do formato ou conteúdo pretendidos, tornou-se necessário proceder a uma análise mais aprofundada dos resultados produzidos. Embora o resultado final tenha sido satisfatório, este processo demonstrou que a geração livre pode acarretar custos adicionais de validação e de processamento posterior. Na outra face da moeda, o dataset gerado por meio de Constraint Decoding apresentou uma estrutura muito mais consistente. Do ponto de vista estrutural, não foram identificadas lacunas relevantes durante a avaliação, visto as restrições aplicadas durante a geração limitarem os resultados possíveis e reduzirem a necessidade de correções posteriores.

Um resultado particularmente interessante surgiu durante a análise da velocidade de inferência. O modelo Mistral 7B apresentou uma velocidade de geração significativamente superior à do Phi-4 14B. No entanto, esta diferença não pode ser diretamente atribuída ao uso de Constraint Decoding, visto os modelos utilizados terem dimensões diferentes e o Mistral 7B ter aproximadamente metade do número de parâmetros do Phi-4.

Quero salientar que esta observação suscitou uma questão que considero interessante para um trabalho futuro. Ao restringir os tokens admissíveis durante a

geração, o Constraint Decoding reduz o conjunto de possibilidades que o modelo pode considerar em etapas específicas. Dependendo da implementação utilizada, esta restrição poderá afetar o custo da geração. No entanto, para determinar se existe efetivamente uma melhoria de desempenho causada pelo Constraint Decoding, seria necessário realizar um estudo controlado, utilizando o mesmo modelo, o mesmo hardware, o mesmo contexto e os mesmos dados, e comparar apenas a utilização, ou não, de restrições. Deixo aqui no ar porque pode ser uma direção interessante a seguir no futuro.

Por fim, foi também apresentada uma documentação técnica que serviu como guia para interpretação e implementação desta técnica.

3.3.3 Conclusão e Futuro

Concluiu-se que os Dados Sintéticos constituem uma técnica cada vez mais importante no desenvolvimento e avaliação de sistemas de Inteligência Artificial. A possibilidade de gerar dados de maneira controlada possibilita a criação de exemplos para diversos fins. Com a evolução dos modelos de linguagem, esta capacidade tornou-se mais acessível, visto os próprios modelos poderem ser utilizados para gerar dados estruturados e não estruturados. No entanto, esta facilidade de geração não dispensa a validação. É possível produzir um grande volume de dados sintéticos, mas a sua utilidade depende da qualidade, diversidade e representatividade dos exemplos gerados.

Considero que esta é uma das áreas do estágio com maior potencial de exploração no que se refere a trabalho futuro. Uma possibilidade seria desenvolver sistemas capazes de criar, analisar e selecionar dados de forma iterativa. Por exemplo, poderiam ser utilizados Agentes que, dentro de um conjunto de regras bem definido, gerassem novos exemplos, avaliassem a respetiva qualidade e eliminassem os que não cumprissem determinados critérios. Este processo poderia formar um ciclo controlado de geração e avaliação, reduzindo progressivamente a necessidade de intervenção manual. Outra possibilidade seria desenvolver sistemas capazes de gerar dados com diferentes níveis de dificuldade. Partindo de um conjunto inicial de exemplos, o sistema poderia analisar as limitações encontradas e gerar novas situações ligeiramente mais complexas. Assim, seria possível construir conjuntos de dados progressivos, em que a dificuldade dos exemplos aumentaria de acordo com critérios previamente definidos.

Gostaria também de deixar uma nota menos positiva sobre esta área. Ao longo do estágio, durante a investigação realizada, constatei que uma parte significativa da documentação e dos trabalhos disponíveis sobre dados sintéticos se encontrava muito direcionada para a sua utilização no treino de modelos de linguagem. Esta concentração dificulta a identificação de abordagens e metodologias orientadas para outros casos de utilização, como a validação de sistemas, a criação de benchmarks ou a avaliação de aplicações baseadas em LLM. Isto acaba por ser um copo meio cheio, pois também pode ser relevante explorar mais essas áreas por não haver tanta informação sobre.

Parte IV - Sistemas Estágio Curricular

4.1 How To RAG (Retrieval Augmented Generation)

4.1.1 Fase de Investigação

No que se refere à aplicação prática, uma das ideias discutidas desde o início do estágio era a construção de um sistema capaz de utilizar a documentação própria da empresa como contexto para um modelo de linguagem. Em vez de depender exclusivamente do conhecimento adquirido durante o treino do modelo, o objetivo era que este pudesse consultar informação específica da organização e utilizá-la para responder às perguntas dos utilizadores. Esta abordagem tem vindo a assumir uma importância cada vez maior no desenvolvimento de aplicações baseadas em Inteligência Artificial. O conceito que permite concretizar esta abordagem é designado por **RAG (Retrieval-Augmented Generation)**. De forma simplificada, um sistema RAG, perante uma pergunta específica, identifica os excertos mais relevantes num conjunto de documentos e fornece essa informação ao modelo de linguagem como contexto.

Esta abordagem resolve um problema bastante evidente. Se houver centenas ou milhares de documentos, não seria eficiente fornecer todos ao modelo sempre que um utilizador fizesse uma pergunta. Além do custo computacional, estaríamos a introduzir uma quantidade significativa de informação irrelevante. Num sistema RAG, os documentos são previamente processados e divididos em unidades mais pequenas, normalmente designadas por **chunks**. Quando o utilizador faz uma pergunta, o sistema pesquisa os chunks mais relevantes e apenas estes são posteriormente utilizados pelo modelo como contexto. Por exemplo, perante a questão "Qual foi o semestre em que a empresa obteve o maior lucro?", não faria sentido fornecer cinco documentos completos sobre diferentes áreas da organização ao modelo. O objetivo seria identificar os excertos que contêm informação sobre os resultados financeiros e fornecer apenas essa informação ao modelo.

Para compreender como esta seleção pode ser realizada, é necessário introduzir um dos conceitos fundamentais dos sistemas RAG: os **Embeddings**.

Os Embeddings constituem uma das principais componentes de muitas aplicações modernas de Inteligência Artificial. Embora a sua utilização tenha adquirido enorme relevância com os atuais modelos de linguagem, a ideia de representar palavras e conceitos por meio de representações distribuídas tem origens anteriores à criação dos Transformers. Um dos fundamentos conceituais desta área está na

Distributional Hypothesis [18], associada aos trabalhos de Harris (1954) e Firth (1957), que estabelece que palavras que surgem em contextos semelhantes tendem a ter relações semânticas semelhantes.

*"Words that occur in similar linguistic contexts tend to have similar meanings.
Equivalently: the meaning of a word is reflected in the distribution of contexts in
which it occurs."*

Distributional Hypothesis Definition

"You shall know a word by the company it keeps."

Firth

Sempre que se pretende utilizar texto como entrada de um sistema de Machine Learning, é necessário transformá-lo numa representação numérica. Existem várias abordagens clássicas para realizar esta transformação, nomeadamente One-Hot Encoding, Bag-of-Words, n-grams e TF-IDF. Embora úteis, estas técnicas apresentam uma limitação importante: apenas representam propriedades lexicais e estatísticas do texto. Por exemplo, conseguem indicar que determinadas palavras aparecem num documento e com que frequência, mas não representam diretamente a relação semântica entre diferentes palavras ou frases.

O TF-IDF (*Term Frequency-Inverse Document Frequency*) é um exemplo particularmente relevante. A sua representação tem em conta a frequência de um termo num documento e a sua importância em relação ao conjunto de documentos. No entanto, palavras semanticamente semelhantes continuam a ser tratadas como entidades distintas.

Um **Embedding** procura ultrapassar esta limitação através da representação de uma unidade de informação - como uma palavra, frase, documento ou, dependendo do modelo, outro tipo de informação - através de um **vetor num espaço contínuo**. Estes valores não são definidos manualmente, mas aprendidos através de algoritmos de Machine Learning. O objetivo é que a representação aprendida capture características relevantes da informação representada.

Esta ideia adquiriu particular importância com os trabalhos *A Neural Probabilistic Language Model* [19], *Word2Vec* [20], *GloVe* [21] e *FastText* [22]. Estes métodos demonstraram que é possível aprender representações vetoriais capazes de capturar relações úteis entre palavras.

No entanto, estes primeiros Embeddings apresentavam uma limitação importante: **eram essencialmente estáticos**. Assim, uma mesma palavra tendia a ter a mesma representação, independentemente da frase em que aparecia. Tal é problemático,

dado que o significado de uma palavra pode depender fortemente do contexto em que é utilizada. Foi neste contexto que surgiram os denominados embeddings contextuais, com trabalhos como o *ELMo* [23] e, posteriormente, *BERT* [24]. A representação deixa de depender apenas da palavra isolada, passando a considerar o contexto em que esta surge.

Esta característica é particularmente importante para um sistema RAG. Ao transformarmos tanto a pergunta do utilizador como os diferentes segmentos dos documentos em vetores, é possível comparar as respetivas representações e identificar os conteúdos mais próximos. Assim, a pesquisa deixa de depender exclusivamente da correspondência exata entre palavras, passando também a considerar relações semânticas aprendidas pelo Encoder (modelo de embedding).

4.1.2 Fase de Aplicação

Após a fase de investigação, procedeu-se à construção de um sistema RAG. Não se pretendia apenas demonstrar o conceito de RAG, mas sim construir uma implementação que permitisse experimentar diferentes estratégias de recuperação, avaliar os respetivos resultados e compreender os principais fatores que influenciam a qualidade das respostas.

Para a implementação, recorreu-se à library **LangChain** [25], que permitiu estruturar os vários componentes do sistema e facilitar a integração entre as diferentes etapas do processo. Embora o sistema tenha sido desenvolvido com base nesta estrutura, procurei compreender o funcionamento de cada componente individualmente, a fim de não tratar o RAG apenas como uma abstração fornecida por uma biblioteca.

Arquitetura do Sistema

O sistema desenvolvido pode ser dividido em várias etapas: Parsing, Chunking, geração de embeddings, armazenamento numa base de dados vetorial, Retrieval, Hybrid Retrieval, Reranking, construção do contexto e geração da resposta pelo modelo de linguagem.

A primeira etapa corresponde ao Parsing.. Nesta fase, os documentos são transformados numa representação que possa ser posteriormente processada pelo sistema. O objetivo é extrair o conteúdo relevante, preservando, tanto quanto possível, a informação necessária para as etapas seguintes.

Após a extração do conteúdo, os documentos são divididos em unidades mais pequenas, denominadas "chunks". Esta fase, designada "chunking", é particularmente importante, uma vez que influencia diretamente a qualidade da recuperação. Unidades de informação demasiado grandes podem introduzir informação desnecessária no contexto fornecido ao modelo, ao passo que unidades de informação demasiado pequenas podem separar informação que deveria ser interpretada em conjunto.

Após a divisão dos documentos, cada chunk é convertido num embedding. Como explicado anteriormente, um embedding representa a informação sob a forma de um vetor num espaço contínuo. Esta representação possibilita a comparação semântica da pergunta do utilizador com os diferentes excertos existentes na base de conhecimento.

No entanto, à medida que o sistema foi sendo desenvolvido, ficou evidente que uma única estratégia de recuperação poderia não ser suficiente. Embora a pesquisa

exclusivamente vetorial apresente vantagens na identificação de relações semânticas, pode haver casos em que a correspondência lexical é importante e em que esta estratégia pode ter dificuldades. Por este motivo, foi implementado o Hybrid Retrieval, que combina diferentes estratégias de pesquisa.

Uma das componentes utilizadas foi a pesquisa vetorial, com base em embeddings, ao passo que a outra abordagem teve por base o BM25, um método de pesquisa lexical que tem em conta a ocorrência e a relevância dos termos nos documentos. A combinação destas duas abordagens permitiu explorar, em simultâneo, as relações semânticas e as correspondências explícitas entre os termos utilizados na pergunta e os termos presentes nos documentos.

Após a primeira fase de recuperação, foi introduzido um Reranker. Ao passo que o sistema de recuperação inicial tem como objetivo obter rapidamente um conjunto de resultados relevantes, o Reranker analisa de forma mais aprofundada a relação entre a pergunta e cada um dos resultados obtidos, permitindo reorganizar os resultados e selecionar os mais relevantes.

Nesta componente foram explorados modelos *Encoder* e *Cross-Encoder*. Um *Encoder* permite transformar informação, como texto, numa representação vetorial que pode posteriormente ser comparada com outras representações. Já um *Cross-Encoder* recebe diretamente o par formado pela pergunta e pelo documento ou *chunk*, processando ambos em conjunto para estimar a sua relevância. A utilização destas diferentes componentes permitiu construir um pipeline em várias etapas, no qual a pesquisa inicial privilegia eficiência e abrangência e o Reranking posterior procura aumentar a precisão dos resultados mais relevantes.

Avaliação do Sistema

A avaliação do sistema constituiu uma parte importante do seu desenvolvimento. Em vez de apenas analisar visualmente as respostas produzidas pelo modelo de linguagem, calcularam-se métricas específicas para avaliar a qualidade da recuperação.

Entre as métricas utilizadas, destacam-se o Recall@K, o Precision@K, o HitRate@K e o Mean Reciprocal Rank (MRR).

O Recall@K permite avaliar a capacidade do sistema para recuperar a informação considerada relevante entre os primeiros K resultados. Por sua vez, o Precision@K avalia a proporção dos resultados recuperados que são efetivamente relevantes. A métrica HitRate@K permite verificar se foi recuperado pelo menos um resultado

relevante entre os primeiros K resultados. Por fim, o MRR tem em conta a posição do primeiro resultado relevante, atribuindo-lhe um valor mais elevado se aparecer numa posição mais alta. Estas métricas possibilitaram a avaliação do comportamento das diferentes estratégias de recuperação e a compreensão do impacto de componentes como o Hybrid Retrieval e o Reranker no sistema.

Além da avaliação da recuperação, foi também necessário avaliar a qualidade das respostas finais produzidas pelo modelo de linguagem. Para esse efeito, foram consideradas métricas como *Answer Relevance* e *Faithfulness*. *Answer Relevance* avalia se a resposta produzida responde efetivamente à pergunta colocada pelo utilizador. A métrica *Faithfulness* avalia se a resposta está fundamentada na informação disponibilizada como contexto, o que é particularmente relevante em sistemas RAG, visto um dos principais objetivos deste tipo de arquitetura ser reduzir as respostas não suportadas pela informação recuperada.

Dados e integração com as fases anteriores

Para o desenvolvimento e avaliação do sistema, foram utilizados documentos fictícios criados especificamente para o efeito. Esta opção permitiu a construção de diferentes cenários de teste, sem recurso a informação real ou potencialmente sensível da organização.

A criação destes dados esteve diretamente relacionada com a fase anterior de Dados Sintéticos. Os exemplos utilizados para avaliação foram produzidos recorrendo às técnicas de geração de dados sintéticos previamente estudadas, nomeadamente Context Engineering e Constraint Decoding. Desta forma, as várias fases começaram a convergir: os dados sintéticos possibilitaram a criação dos exemplos necessários para testar o RAG, ao passo que a engenharia de contexto foi utilizada para orientar os modelos na interpretação e utilização da informação recuperada.

A Engenharia de Contexto assumiu, de resto, um papel importante na fase de geração da resposta. A recuperação da informação correta não é suficiente se o modelo não receber instruções claras sobre como deve utilizar essa informação. Por conseguinte, foi necessário definir o contexto fornecido ao modelo, bem como as suas responsabilidades e as regras a serem seguidas na construção da resposta.

Uma limitação importante desta fase foi o facto de o sistema nunca ter sido aplicado a documentos oficiais ou a informação operacional da empresa. Embora o sistema tenha sido construído e avaliado com dados fictícios, não foi possível validar o seu

comportamento relativamente à documentação real da organização durante o estágio.

Demonstração e Documentação

Como parte final desta fase, foi desenvolvida uma demonstração funcional do sistema que permitiu apresentar todo o processo de forma integrada. A demonstração ao vivo para o professor acompanhante do estágio permitiu apresentar o funcionamento do sistema, desde a introdução da questão até à recuperação da informação e à geração da resposta final.

A demonstração correu bem, comprovando, na prática, o funcionamento da arquitetura desenvolvida e das diferentes componentes que a constituem.

Além do desenvolvimento técnico, foi também criada documentação relativa ao sistema. A referida documentação explica a arquitetura implementada, as diferentes etapas do pipeline, as métricas utilizadas e as decisões tomadas durante o desenvolvimento. Foram ainda identificadas possíveis melhorias futuras, de modo a permitir que o trabalho desenvolvido pudesse servir de base a uma eventual continuação do projeto.

4.1.3 Conclusão e Futuro

Esta fase permitiu-me transitar dos conceitos estudados anteriormente para a construção do primeiro sistema de Inteligência artificial completo pessoal e a nível da empresa. Considerando o âmbito e a duração do estágio, o sistema RAG desenvolvido apresentou um nível de complexidade considerável, uma vez que não se limitou à utilização de embeddings e de uma base de dados vetorial, tendo incorporado diferentes estratégias de recuperação, nomeadamente Hybrid Retrieval, Reranking, Cross-Encoders, bem como mecanismos de avaliação da recuperação e da resposta final.

Um dos principais resultados desta fase foi compreender que a construção de um sistema RAG é um processo composto por várias etapas, sendo que a limitação de uma delas pode afetar significativamente o resultado final. A qualidade dos documentos e a sua segmentação influenciam os vetores de incorporação, que por sua vez influenciam a recuperação. Esta, por sua vez, condiciona o Reranking, e todo este processo determina o contexto que será posteriormente utilizado pelo modelo de linguagem.

No entanto, as possibilidades de evolução dos sistemas RAG são bastante extensas. Uma primeira direção a explorar seria deixar de limitar a fonte de informação aos documentos estáticos. Por exemplo, um sistema poderia recuperar informação diretamente a partir de páginas da internet, hiperligações, bases de dados ou outras fontes de informação estruturada e não estruturada. Desta forma, o sistema deixaria de estar limitado a uma base de conhecimentos previamente construída, podendo trabalhar com fontes de informação mais dinâmicas.

Outra evolução possível seria a utilização de Agentic RAG. Nesta abordagem, o sistema deixa de executar um processo de recuperação fixo e passa a ter maior capacidade de decisão sobre a forma como deve procurar a informação. Por exemplo, um agente poderia decidir quando é necessário realizar uma nova pesquisa, consultar diferentes fontes ou reformular uma pergunta antes de procurar novamente informação.

Outra área que considero interessante explorar é o Corrective RAG (CRAG). Esta abordagem introduz mecanismos de avaliação da qualidade da informação recuperada, permitindo ao sistema determinar se o contexto obtido é suficientemente relevante ou se devem ser realizadas novas operações de recuperação. Desta forma, o sistema passa a ter uma espécie de mecanismo de autocorreção, reduzindo a dependência de uma única tentativa de recuperação.

4.2 Sistema Assobio

O último sistema desenvolvido durante o estágio foi, provavelmente, o de maior complexidade técnica. Surgiu a partir de uma ideia de um colega sobre a análise e auditoria de transcrições de chamadas telefônicas. Inicialmente, o objetivo consistia em receber um ficheiro de áudio de uma chamada, proceder à sua transcrição e, posteriormente, utilizar um modelo de linguagem para analisar a transcrição de acordo com os critérios definidos pelo utilizador.

Este projeto teve uma particularidade importante em relação aos trabalhos anteriores: envolveu diferentes áreas de Inteligência Artificial no mesmo sistema. Além dos modelos de linguagem, foi necessário trabalhar com modelos de reconhecimento automático de fala (ASR), que transformam áudio em texto.

O sistema acabou por evoluir para duas versões distintas. A primeira versão teve como objetivo principal validar o conceito de auditoria automática de chamadas, tendo sido também introduzidos mecanismos de routing de modelos para selecionar diferentes modelos de linguagem e diferentes recursos de hardware. A segunda versão surgiu posteriormente com o intuito de melhorar a qualidade da transcrição, aumentar a capacidade de observação do sistema e adicionar uma nova funcionalidade de interação por meio de um chatbot sobre as transcrições armazenadas.

4.2.1 Sistema Assobio - Versão 1

A primeira versão do sistema Assobio é constituída por um processo relativamente simples: o sistema recebia um ficheiro de áudio, que era processado por um modelo de reconhecimento de voz (ASR) e convertido numa transcrição. Esta transcrição era, posteriormente, analisada por um modelo de linguagem que efetuava uma verificação de acordo com os critérios definidos pelo utilizador.

Uma das características mais interessantes desta primeira versão foi a introdução do routing de modelos, uma ideia sugerida pelo orientador do estágio. O objetivo era evitar a utilização constante do modelo de linguagem mais complexo para todas as transcrições. Consoante a complexidade do conteúdo transcrito, o sistema poderia selecionar um modelo mais adequado para realizar a auditoria.

Esta decisão foi também aplicada à escolha do hardware. Caso a complexidade da transcrição justificasse a utilização de um modelo de maior dimensão, este poderia ser encaminhado para uma GPU. Para casos menos exigentes, seria possível

utilizar recursos computacionais mais modestos, como a CPU. Desta forma, o sistema procurava adaptar os recursos utilizados às características da tarefa, em vez de utilizar sempre a configuração máxima disponível.

Depois do *Model Routing*, o modelo de linguagem selecionado era carregado no hardware correspondente e recebia o contexto definido pelo utilizador. Esta componente permitia personalizar os critérios utilizados durante a auditoria. O utilizador poderia, por exemplo, definir quais os comportamentos a procurar numa chamada, quais as regras a considerar e quais os critérios utilizados para determinar se uma transcrição cumpria os requisitos definidos. O utilizador podendo seleccionar o contexto, o sistema tornava-se universal a qualquer auditoria.

Para executar os modelos de linguagem, utilizou-se o *llama.cpp*, tendo sido disponibilizados **cinco modelos diferentes para utilizar no sistema**. A selecção destes modelos era realizada de acordo com a lógica de encaminhamento de modelos definida.

Desde a primeira versão, foi também atribuída grande importância à observabilidade. Além do resultado final da auditoria, foram registadas várias informações relativas a cada execução, de modo a permitir a compreensão do comportamento do sistema e a identificação de possíveis pontos de melhoria. Os dados registados na base de dados incluíam o contexto fornecido pelo utilizador, o prompt, o áudio original, o áudio após o pré-processamento, a duração do áudio, os tempos de pré-processamento, a transcrição produzida pelo modelo de reconhecimento de fala (ASR) e as respetivas métricas de execução. Do lado do modelo de linguagem, eram igualmente registados: o modelo utilizado; o hardware selecionado pelo *Model Routing*; o resultado da auditoria; o número de tokens processados; o tempo de fase de Prefill; o Throughput da fase de Prefill; o tempo de Decode; o Throughput da fase de Decode; e a latência total.

Esta informação permitiu transformar o sistema numa aplicação observável, na qual não só era possível conhecer a resposta final, como também compreender como esta tinha sido produzida e que recursos tinham sido utilizados.

4.2.2 Sistema Assobio - Versão 1 Fase de Investigação

Após a conclusão da primeira versão funcional do sistema, ficou evidente que a qualidade da auditoria dependia diretamente da qualidade da transcrição. Uma auditoria baseada numa transcrição incorreta poderia conduzir o modelo de linguagem a tirar conclusões erradas, mesmo que o próprio modelo apresentasse um bom desempenho.

Por conseguinte, tornou-se necessário aprofundar o funcionamento dos modelos ASR e compreender os fatores que influenciam a qualidade da transcrição.

Inicialmente, a investigação incidiu sobre o próprio funcionamento digital do áudio. Um sinal de áudio pode ser representado no domínio temporal (*Time Domain Signal*), onde se observa a variação da amplitude do sinal ao longo do tempo. No entanto, esta representação não permite identificar diretamente as frequências que compõem o áudio.

As Transformações de Fourier permitem decompor um sinal e analisar as frequências presentes, obtendo-se assim uma representação no domínio da frequência (*Frequency Domain Signal*). A partir destas representações, pode construir-se um espectrograma que mostra a distribuição das frequências ao longo do tempo. Foi igualmente estudado o **Mel Spectrogram**, uma representação particularmente relevante em sistemas de processamento de áudio, utilizada por diversas arquiteturas ASR. Esta representação imita a forma como o sistema auditivo humano percebe diferentes frequências, permitindo fornecer ao modelo uma representação mais adequada para o processamento de áudio.

Após esta introdução ao processamento de sinais, procedeu-se ao estudo da arquitetura dos modelos ASR. Uma das arquiteturas que ganhou enorme relevância nesta área com a expansão dos Transformers foi o modelo **Whisper**, desenvolvido pela OpenAI. Nos modelos **Encoder-Decoder**, o Encoder recebe a representação do áudio e transforma-a numa representação interna que contém informação relevante para a tarefa de reconhecimento. O Decoder utiliza essa informação para gerar progressivamente a sequência de tokens correspondente à transcrição. Esta explicação é propositadamente muito simples.

No âmbito desta investigação, foram também estudadas outras arquiteturas e abordagens de ASR. Apesar da forte influência dos Transformers, estes não são a única família de soluções existente. Existem modelos e arquiteturas particularmente relevantes quando o objetivo principal é reduzir a latência ou aproximar o funcionamento de um sistema de reconhecimento de fala em tempo real.

Uma parte importante da investigação foi dedicada a modelos ASR em português europeu. Neste âmbito, foi identificado um modelo Whisper adaptado para português europeu, desenvolvido pelo **INESC-ID**, o qual foi posteriormente incluído nos testes realizados na segunda versão do sistema.

Por conseguinte, a escolha do modelo ASR não poderia basear-se exclusivamente na qualidade da transcrição. Era necessário ter em conta, em simultâneo, a qualidade, a latência, os recursos computacionais disponíveis, a língua, as características dos ficheiros de áudio e a possibilidade de utilização em cenários próximos do tempo real.

4.2.3 Sistema Assobio - Versão 1 Fase de Aplicação

Após a implementação inicial, realizou-se uma reunião com o colega responsável pela proposta do sistema, durante a qual foram apresentados três exemplos reais de chamadas telefónicas gravadas. Os exemplos apresentavam características diferentes, incluindo situações com maior presença de ruído e outras com condições acústicas mais favoráveis. Os exemplos permitiram obter uma primeira perceção das condições dos áudios que o sistema teria de processar. Entretanto, surgiu uma nova necessidade: além da auditoria automática, foi proposta a criação de uma interface conversacional que permitisse ao utilizador fazer perguntas sobre as chamadas e as respetivas transcrições.

Nesta fase, foi também construída uma interface gráfica com recurso à biblioteca **Gradio**, que permitiu disponibilizar as principais funcionalidades do sistema através de uma interface web simples. Esta interface facilitou a interação com o sistema durante os testes e as demonstrações, apresentando de forma mais clara o fluxo de processo, desde o carregamento do ficheiro de áudio até à obtenção da respetiva transcrição e auditoria.

A necessidade de disponibilizar uma componente conversacional também marcou a transição para a segunda versão do sistema. Antes de avançar para o chatbot, tornou-se necessário melhorar e avaliar de forma mais rigorosa a componente de transcrição.

Nesta fase, foram estudadas várias técnicas de pré-processamento de áudio e diferentes estratégias de inferência dos modelos ASR. Para avaliar estas abordagens, foi criado um conjunto de dados com base em chamadas telefónicas obtidas a partir de vídeos disponíveis no YouTube.

Foram consideradas quatro métricas principais: *Word Error Rate* (WER), *Character Error Rate* (CER), *Real Time Factor* (RTF) e Latência.

O WER permitiu avaliar a diferença entre a transcrição produzida pelo sistema e a referência ao nível das palavras, ao passo que o CER realizou uma avaliação semelhante ao nível dos caracteres. O RTF possibilitou a análise da relação entre o tempo necessário para processar o áudio e a sua duração, sendo particularmente relevante para avaliar a sua utilização em cenários próximos do tempo real. A Latência complementou esta análise, ao medir o tempo efetivamente necessário para obter a transcrição.

No total foram avaliados nove sistemas/configurações utilizando dois modelos de transcrição: o Whisper Large original da OpenAI e um modelo Whisper adaptado para Português Europeu pelo INESC-ID.

As nove configurações avaliadas foram:

1. *Greedy Decoding*;
2. *Beam Search* com *beam size* igual a 5;
3. *Beam Search* com *beam size* igual a 10;
4. VAD (*Voice Activity Detection*);
5. Demucs;
6. *Noise Reduction*;
7. *Peak Normalization*;
8. Demucs + *Peak Normalization* + aumento controlado do volume;
9. VAD + Demucs + *Peak Normalization* + aumento controlado do volume.

Esta comparação não só permitiu avaliar diferentes modelos, como também o impacto das estratégias de pré-processamento e de Decoding na qualidade e no desempenho da transcrição.

A configuração que obteve os melhores resultados no conjunto de testes consistiu em combinar o **Whisper Large original com o Demucs + Peak Normalization e o aumento controlado do volume**. Por conseguinte, esta configuração foi selecionada para integrar a segunda versão do sistema.

Considero que este teste de desempenho teve uma importância particularmente elevada, na medida em que estabeleceu uma base quantitativa para futuras alterações. Em vez de se introduzirem novas técnicas de pré-processamento ou de se alterar o modelo ASR apenas com base numa perceção subjetiva, passou a existir um conjunto de métricas que poderia ser utilizado para verificar se uma alteração representava efetivamente uma melhoria.

Esta abordagem está também alinhada com a filosofia seguida nas fases anteriores do projeto: antes de otimizar um sistema, é necessário possuir uma forma de medir o seu desempenho.

4.2.4 Sistema Assobio - Versão 2

A segunda versão do sistema teve dois objetivos principais: melhorar a qualidade e a eficiência da transcrição, bem como acrescentar uma componente conversacional que permitisse consultar a informação existente sobre as chamadas.

Para suportar esta nova funcionalidade, a base de dados foi igualmente alargada, de modo a armazenar informação adicional sobre as transcrições e as chamadas correspondentes. O conjunto de modelos disponíveis para o sistema foi igualmente aumentado, tendo passado de cinco para doze modelos com dimensões entre 4 e 31 mil milhões de parâmetros.

A principal alteração arquitetónica consistiu na introdução de uma Camada Semântica entre o modelo de linguagem e a base de dados SQL. Esta camada tem como objetivo permitir que os utilizadores realizem perguntas em linguagem natural, sem que o modelo de linguagem tenha acesso direto à execução de consultas SQL.

A Camada Semântica foi organizada em três componentes principais. O primeiro é o Modelo Semântico, que é responsável por definir o contrato entre o sistema e a base de dados, especificando as operações e colunas que podem ser utilizadas. O segundo componente é o Semantic Query Generation, que interpreta a pergunta do utilizador e produz uma representação estruturada da consulta. Esta componente utiliza um modelo de linguagem fortemente condicionado, produzindo exclusivamente objetos JSON de acordo com a estrutura definida.

Por fim, o Query Compiler transforma essa representação numa consulta executável, recorrendo a uma lógica fortemente limitada por meio de heurísticas e estruturas condicionais previamente definidas.

Esta separação permitiu distinguir claramente a interpretação da linguagem natural da execução das operações sobre a base de dados, limitando a liberdade do modelo na interação com os dados.

4.2.5 Sistema Assobio - Versão 2 Fase de Investigação

No decurso da investigação para a segunda versão, surgiu também uma abordagem aos modelos que considerei particularmente interessante: os denominados **System One Models**. Estes modelos têm a capacidade de tomar decisões ou produzir saídas estruturadas de forma muito rápida, o que os torna ideais para tarefas que não exigem a geração de grandes quantidades de texto.

Esta característica torna-os particularmente interessantes para a componente de geração de consultas semânticas. Em vez de se utilizar um modelo autoregressivo mais complexo e condicionado para produzir uma pequena estrutura JSON, seria possível utilizar um modelo especializado neste tipo de decisão estruturada.

Neste contexto, foi explorado um modelo denominado Laya e foi realizado um pequeno benchmark para avaliar a sua adequação à tarefa. Para construir os dados utilizados nesta avaliação, foi novamente utilizada a abordagem dos dados sintéticos, tendo os exemplos sido gerados recorrendo ao modelo GPT OSS 20B.

Os resultados obtidos demonstraram que o modelo de 7B, utilizado na comparação, apresentou um desempenho ligeiramente superior na tarefa em questão, embora a diferença de qualidade não tenha sido particularmente significativa. A comparação do desempenho mostrou também diferenças de velocidade, o que justificou a análise individual dos modelos e dos respetivos requisitos computacionais.

Embora os resultados não indiquem uma vantagem suficientemente clara para substituir imediatamente a solução existente, o modelo de 7B foi mantido na implementação.

Mais importante do que o resultado concreto deste teste de desempenho foi a conclusão de que este tipo de modelos tem potencial para tarefas de decisão estruturada, sobretudo em comparação com a utilização de modelos de linguagem generalistas mais complexos.

Esta fase evidenciou igualmente uma característica particularmente notória durante o estágio: a rápida evolução da área da Inteligência Artificial. Técnicas e arquiteturas que não estavam integradas no início do estágio surgiram entretanto como alternativas relevantes para os problemas analisados.

4.2.6 Sistema Assobio - Versão 2 Fase de Aplicação

A segunda versão recorre a uma arquitetura baseada em vLLM, executada através do Docker, para disponibilizar os modelos de linguagem necessários ao chatbot. A opção por esta tecnologia teve em conta as características do sistema e os requisitos de inferência identificados durante o desenvolvimento, não pretendendo ser uma escolha universalmente superior a outras soluções.

O modelo utilizado nesta componente foi o Phi 15B, numa versão quantizada com GPTQ de 4 bits. A utilização deste formato de quantização permitiu reduzir os requisitos de memória necessários para executar o modelo localmente, mantendo uma dimensão do modelo adequada ao objetivo pretendido.

O chatbot passou então a permitir que os utilizadores fizessem perguntas em linguagem natural sobre as chamadas existentes na base de dados. A Semantic Layer interpretava a pergunta, transformava-a numa consulta estruturada e compilava-a para uma operação SQL controlada. O resultado dessa operação era utilizado para construir a resposta apresentada ao utilizador.

Nesta versão, a interface construída com o Gradio passou também a assumir um papel mais importante, funcionando como ponto de interação com as diferentes componentes do sistema e permitindo demonstrar, de forma integrada, o funcionamento da solução.

A componente de observabilidade foi igualmente expandida. Para além da informação já recolhida na primeira versão, passaram a ser acompanhados de forma mais detalhada os tempos de pré-processamento do áudio, inferência ASR, *decoding*, latência, *tokens/segundo*, modelo selecionado pelo *Model Routing*, hardware utilizado e métricas relativas à inferência do modelo de linguagem, incluindo tempo de *prefill*, *prefill tokens/segundo*, tempo de *decode* e *decode tokens/segundo*.

Com a evolução do sistema, a base de dados passou também a conter mais informação, permitindo que o *chatbot* realizasse consultas mais completas sobre as chamadas e respetivas transcrições.

No final desta evolução, o Sistema Assobio passou de uma solução inicial focada essencialmente em transcrição e auditoria para uma arquitetura mais completa, combinando processamento de áudio, modelos ASR, *Model Routing*, modelos de linguagem, inferência local, base de dados SQL, *Semantic Layer*, interação conversacional e observabilidade. Esta evolução permitiu aplicar, num único sistema, vários dos conhecimentos adquiridos nas diferentes fases do estágio.

Parte V - Conclusão

*“O céu acabou de deixar de ser o teu limite
Missão dada, missão cumprida, seu tropa de elite”*

Professor Rui Pereira - Metamorfose, pt.2 (Deepak Looper, 2018)

Parte VI - Bibliografia

- *Hugging Face – The AI community building the future.*, <https://huggingface.co/>. Accessed 30 September 2026.
- *Worldmetrics — Facts & Grounding Page*, <https://worldmetrics.org/>. Accessed 30 September 2026.
- *ImageNet Classification with Deep Convolutional Neural Networks*, https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf. Accessed 1 October 2026.
- *PyTorch.org*, <https://pytorch.org/>. Accessed 2 October 2026.
- <https://linguisticspath.com/distributional-hypothesis>. Accessed 3 October 2026.
- *A Neural Probabilistic Language Model*, <https://jmlr.org/papers/v3/bengio03a.html>. Accessed 4 October 2026.
- *LangChain: the open agent platform to own your intelligence*, <https://www.langchain.com/>. Accessed 4 October 2026.
- “CUDA Toolkit - Free Tools and Training.” *NVIDIA Developer*, <https://developer.nvidia.com/cuda/toolkit>. Accessed 2 October 2026.
- “1802.05365v2 [cs.CL] 22 Mar 2018.” *arXiv*, <https://arxiv.org/pdf/1802.05365>. Accessed 4 October 2026.
- “1810.04805v2 [cs.CL] 24 May 2019.” *arXiv*, <https://arxiv.org/pdf/1810.04805>. Accessed 4 October 2026.
- “ggml-org/llama.cpp: LLM inference in C/C++ · GitHub.” *GitHub*, <https://github.com/ggml-org/llama.cpp>. Accessed 2 October 2026.
- “Glove: Global Vectors for Word Representation.” *ACL Anthology*, <https://aclanthology.org/D14-1162.pdf>. Accessed 4 October 2026.
- “Infraestruturas de Portugal – Wikipédia, a enciclopédia livre.” *Wikipédia*, https://pt.wikipedia.org/wiki/Infraestruturas_de_Portugal. Accessed 28 September 2026.
- “Infraestruturas de Portugal – Wikipédia, a enciclopédia livre.” *Wikipédia*, https://pt.wikipedia.org/wiki/Infraestruturas_de_Portugal. Accessed 28 September 2026.
- Krizhevsky, Alex, et al. “AlexNet.” *Wikipedia*, <https://en.wikipedia.org/wiki/AlexNet>. Accessed 1 October 2026.
- “microsoft/phi-4 · Hugging Face.” *Hugging Face*, <https://huggingface.co/microsoft/phi-4>. Accessed 3 October 2026.
- “mistralai/Mistral-7B-Instruct-v0.3 · Hugging Face.” *Hugging Face*, <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>. Accessed 3 October 2026.

- “Quem Somos.” *Infraestruturas de Portugal*,
<https://www.infraestruturasdeportugal.pt/pt-pt/sobre-nos/quem-somos>. Accessed 28 September 2026.
- “Quem Somos.” *Infraestruturas de Portugal*,
<https://www.infraestruturasdeportugal.pt/pt-pt/sobre-nos/quem-somos>. Accessed 28 September 2026.
- “Quem Somos.” *Infraestruturas de Portugal*,
<https://www.infraestruturasdeportugal.pt/pt-pt/sobre-nos/quem-somos>. Accessed 28 September 2026.
- “Quem Somos.” *Infraestruturas de Portugal*,
<https://www.infraestruturasdeportugal.pt/pt-pt/sobre-nos/quem-somos>. Accessed 28 September 2026.
- “Qwen/Qwen3-4B-Base · Hugging Face.” *Hugging Face*, <https://huggingface.co/Qwen/Qwen3-4B-Base>. Accessed 3 October 2026.
- “1706.03762v7 [cs.CL] 2 Aug 2023.” *arXiv*, <https://arxiv.org/pdf/1706.03762>. Accessed 30 September 2026.
- “1607.01759v3 [cs.CL] 9 Aug 2016.” *arXiv*, <https://arxiv.org/pdf/1607.01759>. Accessed 4 October 2026.
- “1301.3781v3 [cs.CL] 7 Sep 2013.” *arXiv*, <https://arxiv.org/pdf/1301.3781>. Accessed 4 October 2026.